BACKEND ENGINEERING REVIEW

startupmanager

Roadmap to Portfolio-Ready and a 10/10 Test Suite

Node.js • Express 5 • PostgreSQL (raw pg) • Jest + Supertest

This document consolidates two full code audits into a single ordered work plan. Every item was verified against the source and re-checked by an independent adversarial pass; anything that did not hold up was removed. Work top to bottom — the order is chosen so each item unblocks the next.

Integration test score	**5.0 / 10**
Suite status	**RED (does not pass)**
Endpoint coverage	**16 / 24 (67%)**
Prior findings closed	**8 fixed - 16 partial - 35 open**
Work to portfolio-ready	**~43 hours (~5-6 weeks @ 2 h/day)**

Read this first

The suite is red because of two typos introduced by the last round of fixes, not because your tests are bad. Three hours of work (items 1-3) puts the score at roughly 6.5 — ahead of where you started. Everything after that is genuine forward progress.

How the priorities are labelled:

- P0 — a reviewer finds these in the first ten minutes. Non-negotiable before the repo is shown to anyone.
- P1 — the difference between "promising" and "junior-ready". This is the bulk of the value.
- P2 — genuinely optional. Strong signal, but no hiring manager will reject you for their absence.

Claims are labelled Best Practice (industry standard, not negotiable), Recommendation (my advice, defensible alternatives exist), or Opinion (taste).

1. Where you stand

Honest assessment before the work plan

The architecture is above the junior bar. The verification discipline is not. That gap is the whole story of this review, and it is worth understanding precisely, because it changes what you should work on.

What is genuinely strong — say these in interviews

- Layered architecture enforced across 26 endpoints with no leakage: no controller imports the database, no repository imports a service.
- Authorization built as composable service-layer primitives in guards/serviceGuard.js — a new route cannot bypass authz by forgetting a middleware.
- Zero SQL injection surface across all five repositories, including the wildcard-tempting ILIKE search.
- A correct transaction with BEGIN/COMMIT/ROLLBACK and client.release() in a finally block — the detail that leaks connection pools in real codebases.
- kickMember uses a data-modifying CTE (WITH deleted AS (DELETE ... RETURNING) SELECT ... JOIN) to delete and return enriched data in one round trip.
- Integration tests against a real Postgres with a production-database safety interlock in cleanDatabase — most seniors do not write that.
- Invite acceptance is correctly triple-scoped: invite id AND startup id AND the caller's own user id.
- No cross-tenant read or write could be constructed against tasks or members. Every query pins startup_id.

What is holding it back

- Verification discipline. A commit titled "fix" turned the suite red and it was not run. This is habit, not knowledge — and it is the one that generalises.
- Finishing. validateUUID in 10 of 11 middlewares. requireUserBeginJoined in 1 of 3. expectAuth in 8 of 14. A guard imported and never written. A reviewer reads that pattern once and calibrates every future claim against it.
- Secrets hygiene. A tracked .env is a reflexive filter at most companies, independent of everything underneath it.
- Operability. No pool error handler, no boot validation, no health check, no rate limiting, no indexes, no runnable migrations. Individually small; collectively they read as "has never had to keep something running."

> **The pattern worth internalising**
> Nearly every finding in this document is a case where the correct pattern already exists somewhere in your own codebase. requirePermission in 3 of 4 task mutations. validateUuid in 10 of 16 middlewares. RETURNING * in one repository function but not its sibling. You know how to do these things. The habit to build is the checklist that applies them everywhere — and the test that fails when you don't.

2. P0 — Blockers

13 hours. Do these before the repo is shown to anyone.

- [x] **1. Restore the API error-code contract** — 1 h — CRITICAL

`middlewares/errorHandler.js:12` · `customErrors.js:5`

customErrors.js sets this.errorCode, but the error handler reads err.code — a property that does not exist on AppError. The || fallback therefore fires on every domain error, so a 403 NO_PERMISSION returns code INTERNAL_SERVER_ERROR. Status codes stay correct, which is exactly why this is invisible without running the suite. It breaks roughly 58 assertions across 14 files.

Fix: Change err.code to err.errorCode. Better: rename the field to code in customErrors.js so both sides use one name and this class of typo cannot recur.

- [ ] **2. Get the suite green, then re-enable the message assertion** — 2.5 h — CRITICAL

`tests/helpers/expectError.js:18` · `guards/serviceGuard.js:15,49` · `16 test files`

Run npm test. Fix the roughly six message drifts it surfaces — most are trailing periods added to guard messages by the last fix pass. Separately: expectError still accepts a message argument that nothing reads, at ~76 call sites. Not asserting exact copy is the right call; leaving a dead parameter that looks like an assertion is not.

Fix: Reconcile the drifts, then either delete the message parameter entirely or make it explicitly optional and documented. Assert status + code as the contract.

- [x] **3. Remove the String() coercion in signup and write normalisation back** — 2 h — CRITICAL

`middlewares/signupMiddleware.js:8-14` · `services/authService.js:14-17` · `utils/validateField.js:9`

String(undefined) is the 9-character string "undefined" — truthy, a string, non-blank, and it matches the username regex. Every validation in the file is therefore satisfied by a completely absent field. The controller then reads the raw body and the service calls .trim() on undefined, producing a 500. Separately, the sanitised values are computed into locals and never written back to req, so nothing is lowercased in the database — and email is a case-sensitive UNIQUE column, so Foo@Bar.com and foo@bar.com are two accounts.

Fix: Delete the String() wrappers and validate the raw values. Normalise after validation and assign back to req.body. Remove the "null" blacklist — it also rejects a legitimate task titled "null". Then add a unique index on lower(email).

- [x] **4. Get secrets out of git and rotate them** — 1.5 h — CRITICAL

`.gitignore` · `.env` · `tests/.env.test`

Both .env and tests/.env.test are still tracked. The .gitignore entry reads test/.env.test while the directory is tests/ — so it matches nothing. And a gitignore rule never untracks a file git is already tracking. With the signing key, anyone can forge a token for any user id and authenticate as them without a password.

Fix: Rotate both JWT secrets and the database password first (assume they are burned). Then git rm --cached both files, fix the pattern to tests/.env.test, purge history with git-filter-repo before any push, and commit a .env.example.

- [ ] **5. Make the repo runnable by a stranger** — 3 h — CRITICAL

`README.md` · `package.json`

The README is eleven lines with no setup, run, migrate or test instructions, there is no start script, and "main" points at a file that does not load dotenv. Nobody but you can start this project — which caps its portfolio value near zero regardless of the code quality underneath.

Fix: README with prerequisites, every environment variable, both databases, the migration step, run and test commands, and a full endpoint table. Add "start": "node server.js". Move nodemon to devDependencies.

- [x] **6. Split validateUUID into strict and optional** — 2 h — HIGH

`utils/validateUuid.js:7` · `inviteUserToStartupMiddleware.js:5` · `updateMemberRole / acceptInvite / declineInvite`

The guard was relaxed to if (input && !isValid) so that an optional assigned_to would pass. That relaxation also applies to 14 required route params, so malformed UUIDs now reach Postgres and produce a 500 where a sibling endpoint returns 400. It also silently neutered the invite middleware, which reads req.params.startupId while the route declares :startupid.

Fix: Restore if (!isValid) and add a separate validateOptionalUUID that returns early only on null/undefined. Fix the startupid casing. Add the three missing call sites. Classify required vs optional at the call site, never implicitly through truthiness.

7. Shorten the access token

0.5 h
HIGH

`token_generate.js:3-9`

The access token still lives 7 days, with the comment "change to 15 minutes before production" committed alongside it. The refresh token has the same lifetime, so the short-access/long-refresh split you were reaching for does not exist. Logout only deletes the refresh row, so a stolen access token stays valid for a week and nothing in the codebase can stop it.

Fix: 15 minutes for access, read from config. Delete the comment — a committed debugging TODO reads as "shortcuts ship to main".

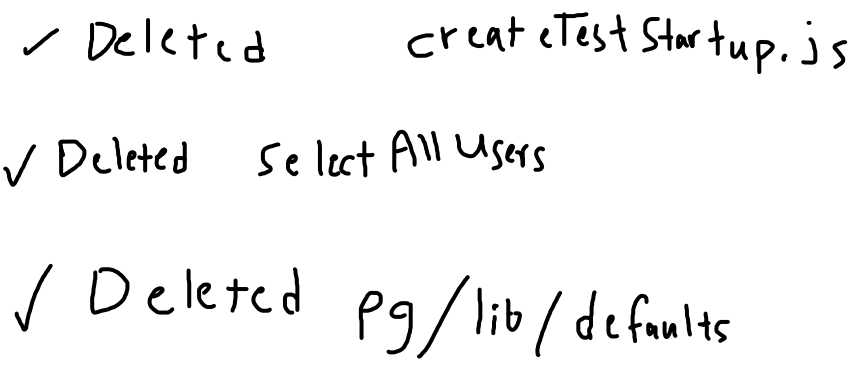
8. Delete the dead code

1.5 h
MEDIUM

`startupService.js:21-31` · `utils/validateParam.js` · `middlewares/getUserStartups.js` · `authRepository.selectAllUsers` · `memberManagmentService.js:33-60` · `tests/helpers/createTestStartup.js`

Dead code is not neutral here: startupService calls requireInvite, which serviceGuard never exports — a security control that was announced in a commit but never written, with an orphaned error code that makes it look implemented. selectAllUsers dumps every user email and is one require away from any controller. createTestStartup.js exports an identifier that does not exist, so requiring it throws.

Fix: Delete all of it, plus the two pg/lib/defaults imports, the duplicate getSpecificMember export key, and the orphaned error code. Add asyncHandler to the two routes missing it, or remove it everywhere — Express 5 forwards async rejections natively.



3. P1 — Backend

18 hours. This is what makes it a junior-ready portfolio.

9. Map Postgres errors at the boundary

2.5 h
HIGH

`middlewares/errorHandler.js` · `app.js`

The error handler has no SQLSTATE mapping and its else-branch hardcodes 500, so malformed JSON returns 500 where body-parser already tagged it 400. This single branch converts most of the remaining validation gaps into correct 4xx responses even before those gaps are individually closed — the highest-leverage item on this page.

Fix: Map 23505 to 409, 23503 to 404/400, 22P02 to 400, 22001 to 400. Honour err.status when it is a 4xx. Handle entity.parse.failed (400) and entity.too.large (413). Add a JSON 404 catch-all so unmatched routes stop returning HTML.

10. Make the process survivable

2.5 h
HIGH

`database/db.js` · `server.js`

database/db.js has no pool error listener. node-postgres emits error on the pool when an idle client dies — a database restart or a network blip — and an unhandled error event on an EventEmitter takes the process down. This is the single most common way a pg-backed Node service dies. There is also no boot-time config validation, so a misspelled secret boots green and fails only on login.

Fix: pool.on('error', ...) plus max, idleTimeoutMillis, connectionTimeoutMillis and ssl. Validate every required env var before app.listen and exit(1) if any is missing. Add GET /health, SIGTERM handling with server.close() then pool.end(), and an unhandledRejection trap that logs and exits non-zero.

11. Add the production middleware stack

1.5 h
HIGH

`app.js` · `services/authService.js`

The entire stack is express.json() plus five routers. POST /auth/login is unauthenticated, unthrottled and bcrypt-backed — simultaneously a credential-stuffing target and a CPU exhaustion vector, since bcrypt occupies a libuv threadpool slot per request.

Fix: helmet, a CORS allowlist, express-rate-limit (strict on /auth/*), and an explicit body size limit. Move the existence check before bcrypt.hash in signup so unauthenticated requests cannot burn CPU.

12. Close the remaining authorization gaps

2 h
HIGH

`services/taskService.js:31,118` · `inviteService.js:91` · `memberManagmentService.js:38,82,113`

assigned_to is validated for UUID shape but never for startup membership, on both the create and update paths — so an owner of startup A can assign a task to a user who only belongs to B, leaking that stranger's id to every member of A. Separately, two requirePermission calls omit the errorMessage argument, producing an AppError with an empty message that the handler renders as "Internal server error" on a 403.

Fix: Call requireUserBeginJoined(startupId, assignedTo) on both task write paths — the guard already exists and is exported. Supply errorMessage at both sites and give the parameter a default. Swap requireJoining for requireUserBeginJoined where the subject is the target user, and move it after requirePermission so a non-owner learns nothing.

13. Fix the schema: constraints, indexes, runnable migrations

3 h
HIGH

`migrations/` · `package.json`

startup_users has no UNIQUE (startup_id, user_id), although invites has exactly that constraint — so uniqueness rests on a check-then-act race across two insert paths, and duplicate rows make getUserRole non-deterministic. There are zero indexes in eleven migrations, yet startup_users is queried twice on every authorized request. Migration 008 adds a NOT NULL column with no default, so it aborts on any non-empty table.

Fix: Add the UNIQUE constraint and indexes on the four foreign keys used in WHERE clauses. Rewrite 008 as nullable, backfill, then set NOT NULL. Add IF NOT EXISTS to the three CREATE TYPE statements. Add a migrate script and a schema_migrations table — or drop node-pg-migrate, which is currently an unused runtime dependency.

14. Finish input validation

2 h
HIGH

`createStartupMiddleware` · `createNewTaskMiddleware` · `updateTaskMiddleware` · `signupMiddleware` · `memberManagmentRepository.js:13`

Only the three signup fields have length bounds. startups.name, tasks.name, users.email and tasks.description are all bounded VARCHARs with no app-layer guard — and migration 011 narrowed tasks.description from TEXT to VARCHAR(255) without adding one, silently creating a new unvalidated-input-to-500 path. There is still no email format check anywhere, so "12313123" is a valid email address.

Fix: One validateLength(field, max, message) helper applied at all six sites. A real email shape check. Escape % and _ in the ILIKE search value and require two non-wildcard characters — parameterisation prevents SQL injection but not wildcard injection.

15. Make repositories report what actually happened

2.5 h
HIGH

`startupRepository.js:80,89` · `inviteRepository.js:28-51,59,102` · `taskRepository.js:11-23,71-83`

Several repository functions return a hardcoded English success message with the query result assigned and never read, so a zero-row delete reports success. Three invite writes return rows[0] from statements with no RETURNING — always undefined — so the controller sends a 200 with an empty body, which a client's res.json() throws on. tasks.updated_at is never written by any UPDATE and there is no trigger, so it permanently equals created_at.

Fix: RETURNING * and return rows[0] rather than hand-assembling responses from the arguments. Check rowCount and throw a 404 when it is zero. Set updated_at = NOW() in the UPDATE. Use rows[0]?.role ?? null in getUserRole so requirePermission fails closed.

16. Fix the list queries

1 h
MEDIUM

`startupRepository.js:63-73` · `taskRepository.js:27` · `memberManagmentRepository.js:21` · `inviteRepository.js:20`

getUserStartups does SELECT * across a join, so startup_users.id overwrites startups.id and GET /startups returns the wrong id — a client cannot navigate from the list to the detail page. Six list queries have no ORDER BY, and Postgres gives no ordering guarantee, so pagination can repeat or skip rows and a task board visibly reshuffles after any edit.

Fix: Name the columns explicitly on every joined query. Add ORDER BY with a unique tiebreaker (for example ORDER BY created_at DESC, id) to all six.

17. Finish the token lifecycle

1.5 h
HIGH

`token_generate.js:13-21` · `services/authService.js:113-137` · `repositories/authRepository.js:29,59-65`

The refresh payload is deterministic to the second, so two logins within the same second produce byte-identical tokens and violate the UNIQUE constraint — a 500 on a correct password. isRefreshTokenValid has no expiry predicate, refresh never rotates, and expired rows are never pruned. Refresh also re-signs claims lifted from the old token rather than re-deriving identity from the database.

Fix: Add a jti (crypto.randomUUID()) to the payload and store that rather than the opaque token. Add AND expires_at > NOW() to the validity check. Rotate on refresh — delete the old row and insert the new one in one transaction — and re-derive the user from the database.

18. Resolve the ownership dead end

2 h
MEDIUM

`services/startupService.js:37-47` · `middlewares/updateMemberRoleMiddleware.js:15`

An owner pressing "leave" silently cascade-deletes the entire startup — every task, membership and invite — and returns 200 with a success message. Ownership can never be transferred, because the role whitelist forbids setting anyone to owner. So an owner has no exit that preserves the startup. "What happens when the founder leaves?" is a question you will be asked.

Fix: Return 409 with guidance pointing at DELETE /startup/:id, which already gates correctly on owner permission. Add a transactional ownership-transfer endpoint. Also decide PATCH semantics: the middleware requires all three task fields while the SQL uses COALESCE for partial updates.

4. P1 — Tests

12 hours. Moves the suite from 5.0 to roughly 8.

19. Make fixtures unique by construction

4 h CRITICAL

`tests/helpers/createOwnedStartup.js:8-13` · `createdJoinedStartup.js` · `all 36 zero-arg call sites`

The composite helpers hardcode identities — createOwnedStartup always uses test@gmail.com — against UNIQUE email and user_name columns. Calling the same helper twice inside one test therefore throws a duplicate-key error, and that is exactly what a cross-tenant test needs: two startups with two different owners. This one constraint is why the entire cross-tenant category is empty. The low-level createTestUser(overrides) already does this correctly; the pattern just did not carry upward.

Fix: Give every composite helper an overrides parameter with uuid-based defaults. Then write the cross-tenant tests: for every list endpoint assert both the length and that no returned id belongs to the other tenant; for single-resource endpoints, request a task from startup B while authenticated as a legitimate member of startup A and expect 404.

20. Cover the untested subsystems

3.5 h HIGH

`tests/integration/ — invites, kick, refresh, profile, users`

The entire invite subsystem — four endpoints, six error branches, and the only transaction in the codebase — has zero coverage. That is precisely why the invite middleware's param typo, its missing validators and its missing errorMessage all survived a full fix pass. Kick is destructive and owner-only with no tests at all.

Fix: Invite lifecycle (invite, accept, decline, duplicate 409, accepting someone else's invite). Kick: happy path with a database assertion, self-kick, non-owner. POST /auth/refresh: valid, revoked, malformed. Then GET /profile and GET /users.

21. Exercise the admin role and the missing case types

2 h HIGH

`tests/helpers/addToStartupWithRole.js` · `tests/integration/`

No test ever issues a request as an admin — addToStartupWithRole has zero importers, so requirePermission(["owner","admin"]) is only ever executed with an owner or a worker. Narrow all three task sites to ["owner"] and nothing fails. Conflict, invalid-enum and empty-collection categories are all empty; 17 of 33 error codes are asserted by nothing.

Fix: Admin as an actor on all three owner-or-admin task endpoints, plus 403 for admin on owner-only operations. Duplicate email, duplicate username and duplicate invite (409). Invalid enums that are the right type — status "banana", role "superadmin". Empty-collection responses on every list endpoint.

22. Verify database state on the remaining writes

1.5 h HIGH

`update_task.test.js` · `update_member_role.test.js` · `login.test.js` · `create_startup.test.js:40`

PATCH task and PATCH role assert only the response body — update_task.test.js has no db import at all. Several assertions also verify only the echoed request values, because the repository hand-assembles responses from its arguments rather than from RETURNING. And no test ever uses a token returned by POST /auth/login: the helper signs { id } while production signs { id, email }, so 62 call sites forge a token the API never issues.

Fix: Query the database after every write. Assert the ownership row after POST /startup and the cascade after DELETE. Add a login round-trip that reuses the returned access_token against a protected route — that is the regression test for the password-in-JWT fix that was never written.

23. Remove the assertions that cannot fail

1 h MEDIUM

`leave_startup.test.js:138` · `get_startup.test.js:28` · `get_all_tasks.test.js:32` · `get_all_members.test.js:20`

leave_startup asserts that tasks are empty in a test whose setup creates no tasks — drop the cascade from the schema and it still passes. get_startup uses typeof x === "object", which is true for null. Two list tests assert only body[0] shape, so they would pass if the endpoint returned every row in the database — useless as the tenant-scoping check a list endpoint most needs.

Fix: Seed a task before asserting the cascade. Replace typeof checks with Array.isArray plus length and value assertions. Fix the four test titles that describe the wrong verb or the wrong actor.

24. Harden the test infrastructure

1 h
MEDIUM

`tests/helpers/expectAuth.js:6` · `jest.config.js` · `6 test files still hand-rolling auth`

expectAuth is declared async while it is used as a synchronous test registrar — it works only because its body contains no await. Add one and 16 tests silently vanish while the suite stays green. jest.config.js has no maxWorkers, so the entire suite's correctness rests on a flag in package.json; running npx jest directly gives you parallel workers all truncating one shared database.

Fix: Drop the async. Add maxWorkers: 1, testMatch and testTimeout to the config. Adopt expectAuth in the remaining six files and delete the twelve hand-copied duplicates.

5. The path to a 10/10 test suite

What each tier actually requires

A 10/10 suite means one thing: every behaviour your code has is pinned by an assertion that would fail if that behaviour broke. No unfalsifiable assertions, no untested paths, no green tests defending bugs. The score is only a proxy for that.

Tier	What it takes	Items	Hours
5.0 -> 6.5	Green suite. The error-code contract restored, the String() coercion removed, and the dead message parameter dealt with. No new tests required.	`1-3`	`~5.5`
6.5 -> 8.0	Unique fixtures, then the cross-tenant category. All 26 endpoints covered. Admin exercised as an actor. Conflict, invalid-enum and empty-collection cases. Database verification on every write path.	`19-24`	`~12`
8.0 -> 10	Authorization matrix test. Mutation testing. Parallel-safe isolation. A single factory module. Coverage floor enforced in CI.	`below`	`~10`

The two moves that actually earn the last two points

1. An authorization matrix test

One table enumerating role x operation x expected status — owner, admin, worker and non-member against create, read, update, delete, assign and kick — asserting the entire grid at once. The reason this matters: updateTask missing its permission guard was not a one-off bug, it was an instance of a class ("guard applied to some call sites, not all"). Case-by-case tests catch instances. A matrix catches the class, because a missing guard appears as a hole in the grid. It is also the strongest single artifact you could produce from this project for an interview.

```
const MATRIX = [
  { role: 'owner',  op: 'updateTask', expect: 200 },
  { role: 'admin',  op: 'updateTask', expect: 200 },
  { role: 'worker', op: 'updateTask', expect: 403 },   // <- the bug you shipped
  { role: 'none',   op: 'updateTask', expect: 403 },
  // ... every role x every guarded operation
];

test.each(MATRIX)('$role $op -> $expect', async ({ role, op, expect: status }) => {
  const ctx = await seedStartupWithRole(role);
  const res = await OPERATIONS[op](ctx);
  expect(res.status).toBe(status);
});
```

2. Mutation testing with Stryker

This is the only objective measure of suite quality that exists. Stryker mutates your source — flips > to >=, deletes a guard call, replaces ["owner","admin"] with an empty array — reruns the suite, and reports which mutants survived. Every survivor is a hole where your code could break and no test would notice. A surviving mutant on requirePermission would have flagged the updateTask gap on day one, with no human reviewing anything.

```
npm install --save-dev @stryker-mutator/core @stryker-mutator/jest-runner
npx stryker run

# Read the survivors. Each one is a test you should have written.
```

Supporting work for the 10 tier

- Parallel-safe isolation: give each Jest worker its own database (startup_manager_test_${process.env.JEST_WORKER_ID}) selected in database/db.js, then drop --runInBand. Note that the tempting alternative — wrapping each test in a transaction and rolling back — does not work here, because supertest fires a real HTTP request and the handler takes its own client from the pool.
- A single factory module replacing the seven name-colliding helpers.
- Coverage reporting in CI with a floor that can only ratchet upward.
- One test per SQLSTATE mapping, so the error-boundary work can never silently regress.

> **Where the return on effort actually bends**
> Recommendation: job-readiness saturates around 8/10. Everything from 5.0 to 8 makes you measurably more hireable. The 9-to-10 tier — mutation testing and parallel isolation — barely moves hireability, but it is strong interview material precisely because most mid-level developers have not done it. Worth doing only after P0 and P1 are complete.

6. Coverage gaps

16 of 24 endpoints tested. The untested third holds the destructive ones.

Endpoint	Tested	Why it matters
`POST /auth/refresh`	NO	Was completely broken and nobody noticed, because access tokens last 7 days so the client never refreshes.
`DELETE .../members/:id/kick`	NO	Destructive and owner-only. Zero tests.
`POST .../users/:id/invite`	NO	Middleware validates nothing; a nonexistent user id produces a 500 leaking the FK constraint name.
`POST /invites/:id/accept`	NO	The only transaction in the codebase. Returns an empty body.
`POST /invites/:id/decline`	NO	Returns an empty body.
`GET /invites`	NO	Never lists invites you sent — only ones you received.
`GET /users`	NO	Full directory dump; the search guard is bypassable with wildcards.
`GET /profile`	NO	Reads identity off the token rather than the database, so it is stale by construction.
`PATCH .../tasks/:id`	PARTIAL	No database assertion at all — the db import is absent.
`PATCH .../members/:id/role`	PARTIAL	db imported but never used. The owner-only guard has no coverage.
`POST /auth/login`	PARTIAL	No status assertion; the issued token is never used against a protected route.
`POST /startup`	PARTIAL	Counts startups but never checks that the ownership row was created.

Case-type coverage across the whole suite: cross-tenant 0 · admin-as-actor 0 · conflict/409 0 · invalid-enum 0 · empty-collection 0 · pagination 0 · error codes asserted 16 of 33 · database state verified on 7 of 10 tested write paths.

7. P2 — Optional

Strong signal, but no hiring manager will reject you for their absence.

Item	Why it is worth doing	Value
GitHub Actions + Postgres service	The highest signal per minute available. Right now nothing proves your test files pass; from outside the repo they carry almost no weight.	HIGH
docker-compose for Postgres	Directly solves "the reviewer cannot run this", which is the project's biggest presentation problem.	HIGH
ESLint + Prettier	Would have caught roughly fifteen dead imports, the duplicate export key and the unused variable in the invite middleware in one pass.	HIGH
Deploy it, URL in the README	Turns an abstract repo into something a reviewer can click. Blocked today by the missing start script and hardcoded port.	HIGH
Pagination on list endpoints	GET /startups returns a bare top-level array, which cannot gain pagination later without a breaking change.	MEDIUM
Request-id middleware	Return the id in the 500 body: fully traceable for you, zero internals leaked to the client.	MEDIUM
Unify the name/title contract	One field currently has three shapes across create and read. Cheap now, while the only client is unwritten.	MEDIUM

Item	Why it is worth doing	Value
OpenAPI or Postman collection	Only worth it once the README endpoint table exists.	LOW
LICENSE, dependency hygiene	nodemon is a runtime dependency. Two minutes.	LOW

8. Timeline

At two focused hours per day

Milestone	Contents	Hours	Calendar
Safe to show anyone	P0 items 1-8. Suite green, secrets rotated, error contract honest, repo runnable.	13	**~2 weeks**
Junior-ready	P1 backend 9-18 and tests 19-24. Test suite at roughly 8.	30	**~4 weeks**
Portfolio-strong	P2: CI, deploy, docker, lint.	~10	**~1-2 weeks**
10/10 test suite	Authorization matrix, mutation testing, parallel isolation.	~10	**optional**

Total to junior-ready: roughly 43 hours, or about five to six weeks allowing for the days you do not get to it. Start applying at the four-week mark — while you are still improving, not after everything is perfect. Applying is practice, and the interview pipeline itself takes another one to three months.

> **The work is unusually front-loaded**
> P0 alone is 13 hours and it removes every disqualifier. After it, the repo boots for a stranger, the suite is green, the error contract is honest, and there are no secrets in git. That is the difference between a reviewer closing the tab in fifteen minutes and a reviewer reading your architecture. If you have to stop somewhere, stop there.

9. Working rules

Habits, not knowledge. These are what actually changed the outcome last round.

1. A red suite means the commit does not happen

Every regression in the last review — all five of them — would have been caught by running npm test before committing. In seconds. For free. The detector was already installed; it was switched off. This single rule is worth more than any individual item in this document.

2. Never weaken a test to make it pass

Commenting out an assertion instead of reconciling the drift it caught is the specific thing a hiring manager probes hardest for. If an assertion fails, either the code is wrong or the assertion is wrong — decide which, out loud, and fix that one.

3. When you fix something, grep for its siblings

validateUUID in 10 of 11 middlewares. requireUserBeginJoined in 1 of 3. expectAuth in 8 of 14. requirePermission with a message in 6 of 8. Every one of those is a fix that was applied where the bug was noticed rather than everywhere the pattern lives. After each fix, search the codebase for the same shape and finish the job.

4. Normalise, then validate, then store

Any field with more than one spelling of the same value — emails, usernames, URLs — needs a canonical form before it reaches validation or the database. Otherwise your UNIQUE constraints are decorative and users get locked out of their own accounts.

5. Know what the framework already guarantees

Express only runs a route's chain when every :segment matched at least one character, so a presence check on a route param can never fire. Twenty-four such checks existed. The danger was never the wasted lines — it was that they displaced the format check that was actually missing.

6. Label your own claims

Best Practice, Recommendation, or Opinion. Doing this in code review, in pull requests and in interviews signals that you know the difference between an industry standard and a personal preference. Very few juniors do it, and it is immediately noticeable.

Next action: change err.code to err.errorCode in middlewares/errorHandler.js:12, then run npm test.

That single character is holding roughly 58 assertions hostage. Everything else in this document is downstream of getting the feedback loop back on.